8-3. Fitting a GLM

- In general, to fit a GLM we need an iterative procedure.
- Iterative weighted least squares can be used to obtain estimates (MLEs) for $\boldsymbol{\beta}$
- Use appropriate computer software, e.g. in `R` use
  `glm(y ~ x, ...)`
  See examples below for more details.

Example 8.1

Recall the data of example 8.1: numbers of stressful events reported by a sample of people for the previous 18 months. We will fit a GLM with Poisson response and log link.

First, read the data into R.

```
> x = 1:18
> n = c(15, 11, 14, 17, 5, 11, 10, 4, 8, 10, 7, 9,
11, 3, 6, 1, 1, 4)
```

Fit the model using the `glm` command.

```
> stressmodel = glm(n ~ x, family = poisson)
```

Note that the `family = poisson` subcommand assumes by default a **log link**. We can specify a different link if appropriate.

(Take a look at R help using `?glm`)

```
> stressmodel=glm(n~x, family=poisson)
> summary(stressmodel)

Call:
glm(formula = n ~ x, family = poisson)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.9886  -0.9631   0.1737   0.5131   2.0362

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  2.80316    0.14816  18.920  < 2e-16 ***
x           -0.08377    0.01680  -4.986 6.15e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 50.843  on 17  degrees of freedom
Residual deviance: 24.570  on 16  degrees of freedom
AIC: 95.825

Number of Fisher Scoring iterations: 5
```

The fitted model is

$$\begin{cases} N_i & \sim & \text{Poisson}\,(\mu_i) \\ \text{with } \hat{\mu}_i & = & \exp\left(2.80316 - 0.08377 x_i\right) \end{cases}$$

So, for instance, 6 months prior to interview, $x = 6$ and the fitted value of expected number of stressful events is

$$\hat{\mu}_6 \quad = \quad \exp(2.80316 - 0.08377 \times 6) \quad = \quad 9.98$$

Fitted values can be obtained in R using

```
> fitted.values(stressmodel)
```

and residuals (by default, *deviance residuals* — see later) using

```
> residuals(stressmodel)
```

We can also calculate confidence intervals for the coefficients in the usual way.

eg using `summary` output, a 95% CI for the slope parameter b is

$$\begin{aligned} \hat{b} \pm z_{0.025} \times s.e.(\hat{b}) &= -0.08377 \pm 1.96 \times 0.01680 \\ &= (-0.117, -0.05) \end{aligned}$$

Alternatively, use the `R` command `confint()`

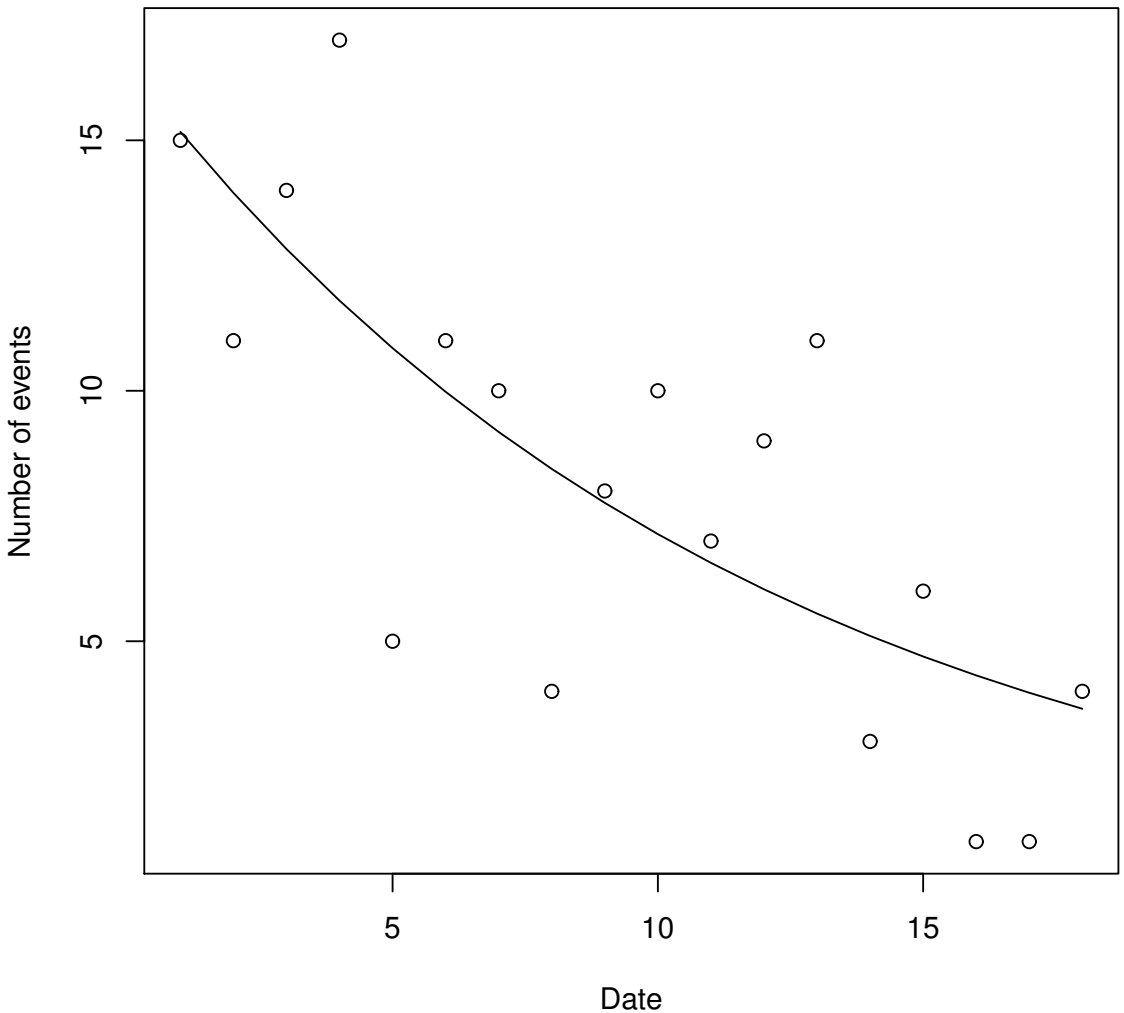
```
> confint(stressmodel)

Waiting for profiling to be done...
                 2.5 %      97.5 %
(Intercept)  2.5044611  3.08575752
x           -0.1172036 -0.05125814
```

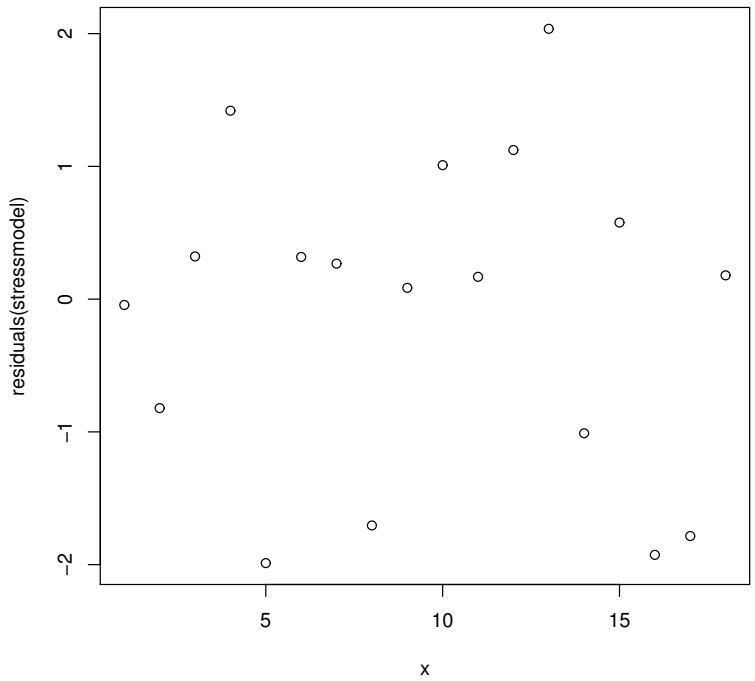
Check `?confint` for details.

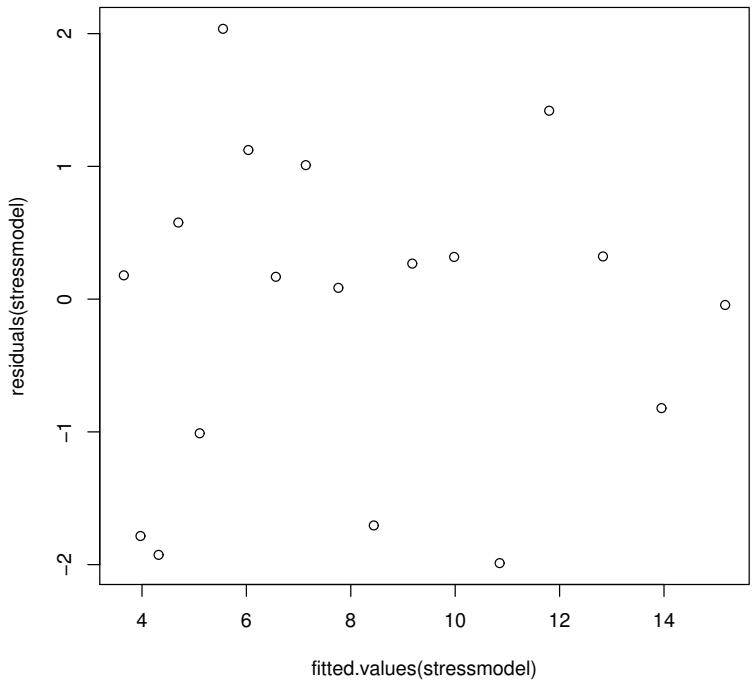
The fitted curve can be drawn with the data using

```
> plot(n~x, xlab="Date", ylab="Number of events")
> lines(fitted.values(stressmodel))
```



```
> plot(residuals(stressmodel)~x)
> plot(residuals(stressmodel)~fitted.values(stressmodel))
```





Residuals (deviance residuals) look randomly distributed — no obvious problem with the model.

Example 8.3

- 10 groups of subjects (insects) are exposed to different doses of a drug.
- The response variable is *propdead*, the proportion of subjects in group killed ($y = d/n$), whilst the explanatory variable is *dose*, the dose of drug administered (in suitable units).

Group	Group size (n)	Number killed (d)	Proportion (y)	Dose (x)
1	64	7	0.1094	3.689
2	66	13	0.1970	3.912
3	59	15	0.2542	4.094
4	62	25	0.4032	4.248
5	55	27	0.4909	4.382
6	60	43	0.7167	4.500
7	63	52	0.8254	4.605
8	57	51	0.8947	4.700
9	60	57	0.9500	4.787
10	61	61	1.0000	4.868

Model: suppose that number killed (D_i) in group of size n_i satisfies

$$D_i \quad \sim \quad Bin\left(n_i,\, \pi_i\right),$$

where the D_i are independent.

So proportion killed $Y_i = D_i/n_i$ in group of size n_i satisfies

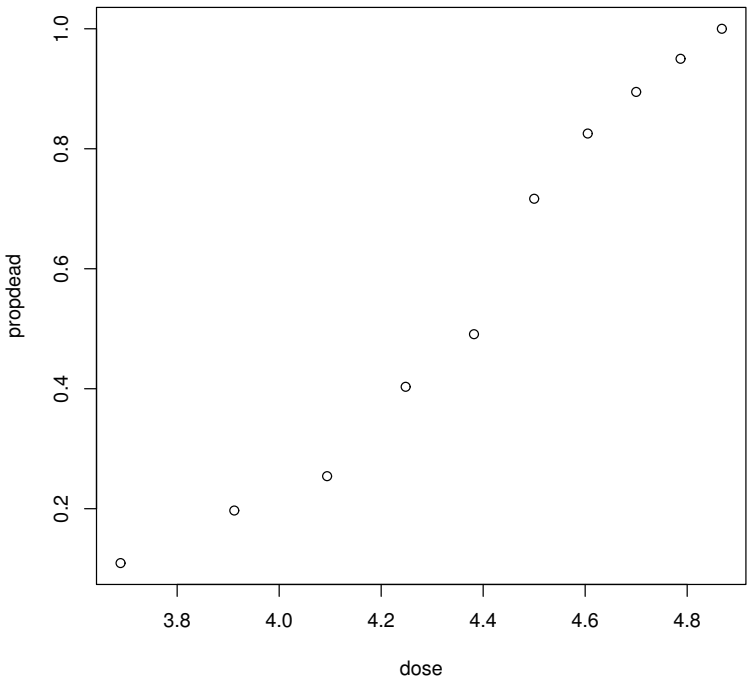
$$n_i Y_i \quad \sim \quad Bin\left(n_i,\, \pi_i\right)$$

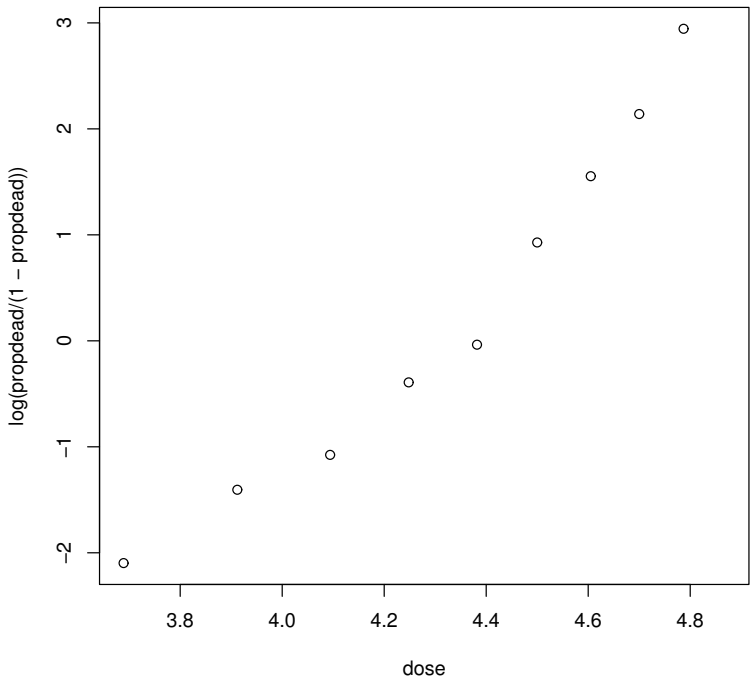
Choose to use logit link:

$$\log\left(\frac{\pi_i}{1-\pi_i}\right) \quad = \quad \eta_i \quad = \quad b_0 + b_1 x_i$$

where b_0 is the intercept and b_1 is the dose effect.

```
plot(propdead ~ dose)
plot(log(propdead/(1-propdead)) ~ dose)
```





Plotting *propdead*, relationship appears non-linear (sigmoid).
Becomes more linear when logit transformation is applied.
We will proceed with a linear predictor (but may need to revise later).

One way to apply logistic regression in R (see `?glm`):

```
drugmodel = glm(propdead ~ dose, weights = groupsize, family = binomial)
```

Note that in R we

- declare the *proportion* of "successes" (here, proportion of subjects killed) as the response
- include the group sizes as a set of weights
- use binomial distribution
- for binomial, logistic is the default link function [otherwise use option `family = binomial(link = "logit")`]

```
> summary(drugmodel)

Call:
glm(formula = propdead ~ dose, family = binomial, weights = groupsize)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-1.8635  -0.5465   0.2424   1.0777   2.7251

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept) -20.2724     1.5956  -12.71   <2e-16 ***
dose          4.7331     0.3661   12.93   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 298.067  on 9  degrees of freedom
Residual deviance:  17.234  on 8  degrees of freedom
AIC: 57.3

Number of Fisher Scoring iterations: 4
```

The fitted model is

$$\begin{cases} D_i & \sim & \text{Binomial}\,(n_i,\, \pi_i) \\ \text{with logit}(\hat{\pi}_i) & = & -20.2724 + 4.7331 x_i \end{cases}$$

So, for instance, for dose $x = 4.0$ the linear predictor is

$$\hat{\eta} = -20.2724 + 4.7331 \times 4.0 = -1.34$$

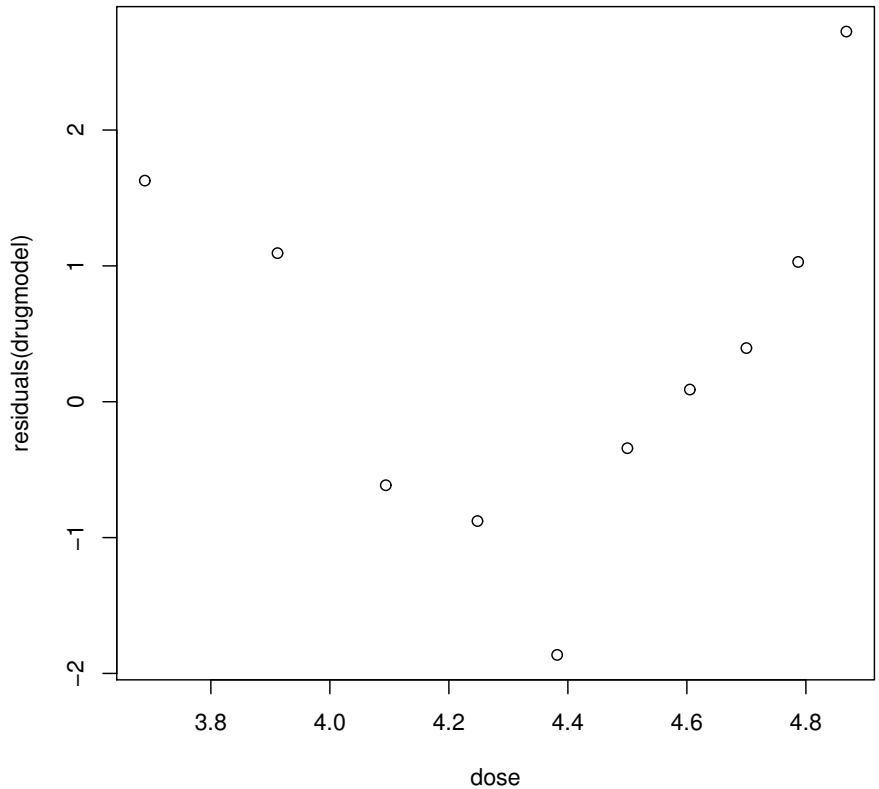
Now $\eta = \log\left(\frac{\pi}{1-\pi}\right)$, and so (inverting logistic function) the fitted proportion killed for dose $x = 4.0$ is

$$\hat{\pi} \quad = \quad \frac{\exp(\hat{\eta})}{1+\exp(\hat{\eta})} \quad = \quad \frac{\exp(-1.34)}{1+\exp(-1.34)} \quad = \quad 0.2075$$

Or directly in R (see `?predict.glm`):

```
> predict(drugmodel,data.frame(dose=4),type="response")
0.2075261
```

```
> plot(residuals(drugmodel) ~ dose)
```



Plot of deviance residuals shows that the model needs to be improved.

To obtain Pearson residuals (see later), would use

```
residuals(drugmodel, type="pearson")
```